

Web Application Vulnerability Scanning with OWASP ZAP

This lab demonstrates **dynamic application security testing (DAST)** using OWASP ZAP against a deliberately vulnerable web application.

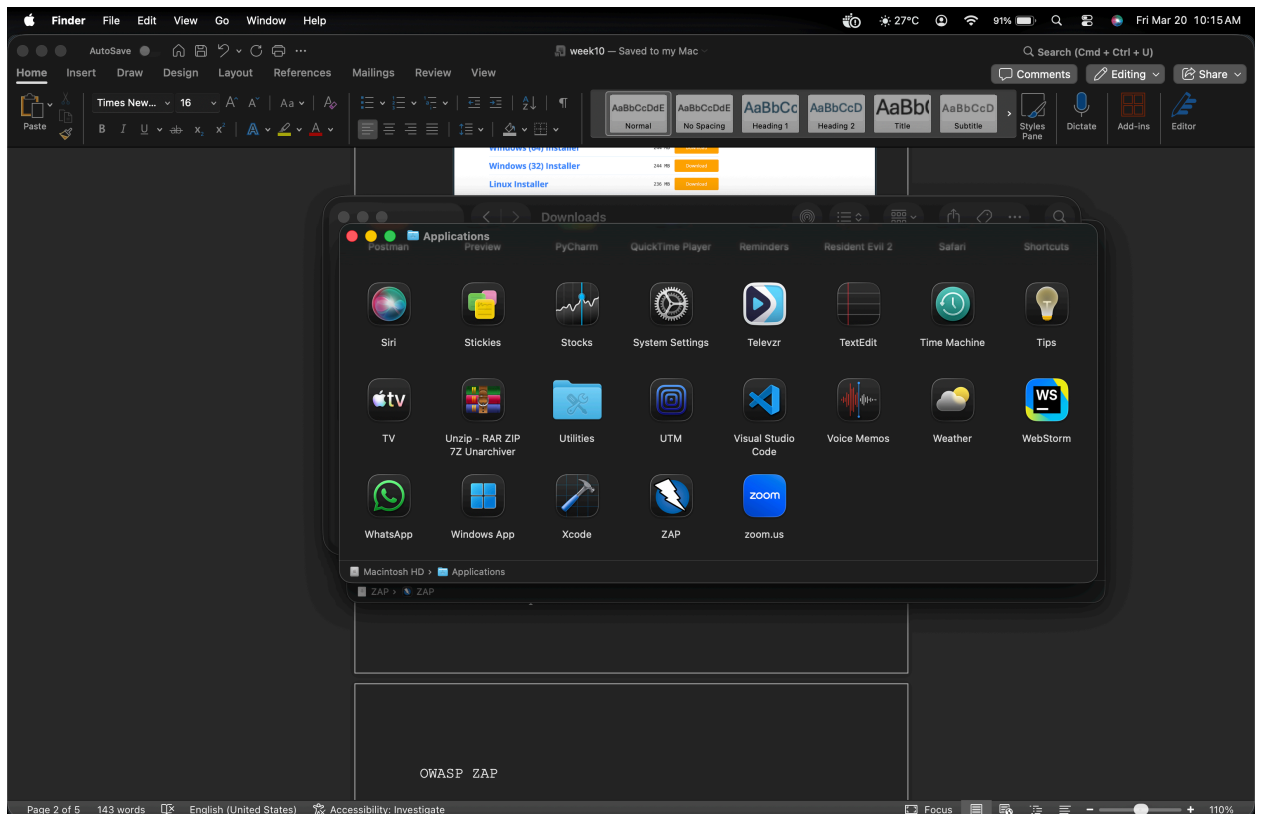
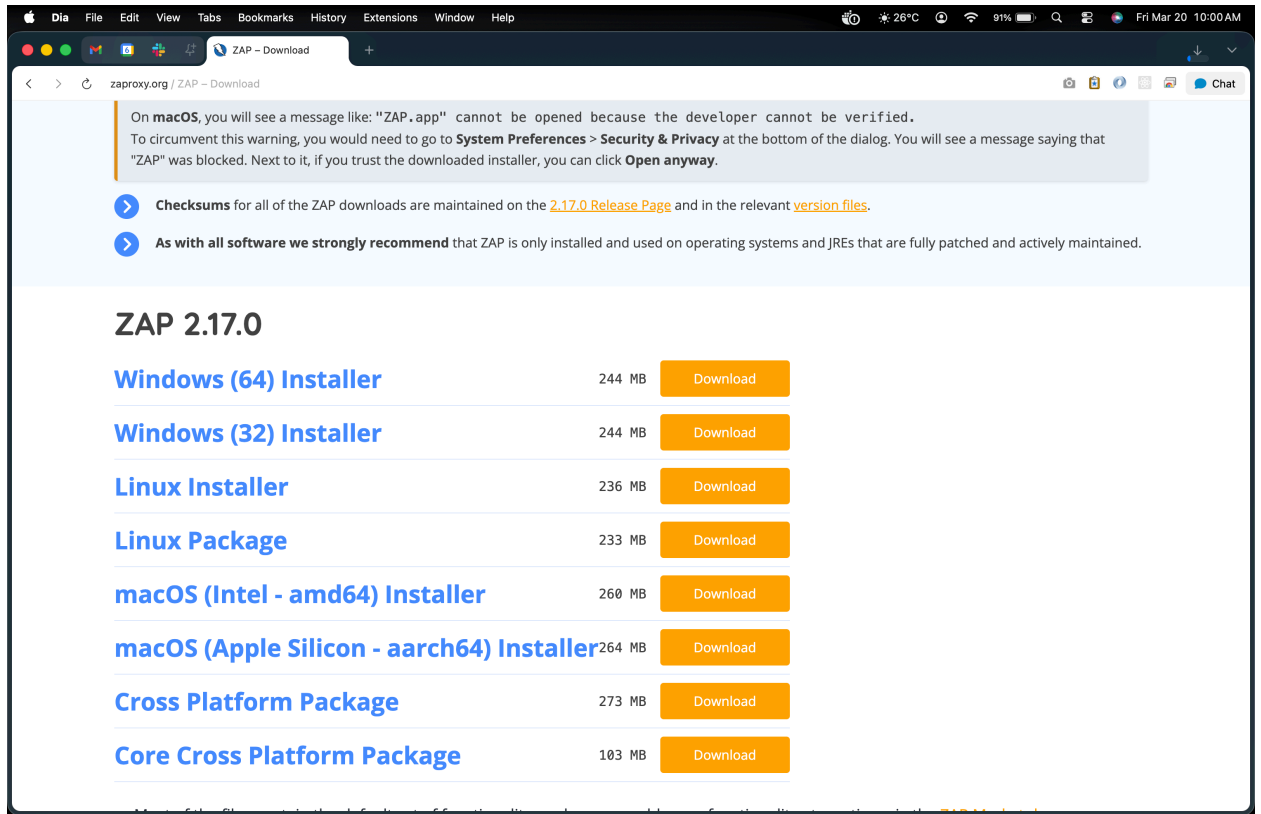
Objective

- Perform automated vulnerability scanning.
- Identify common vulnerabilities (XSS, SQL Injection, etc.).
- Interpret alerts and generate a report.

Lab Environment Setup

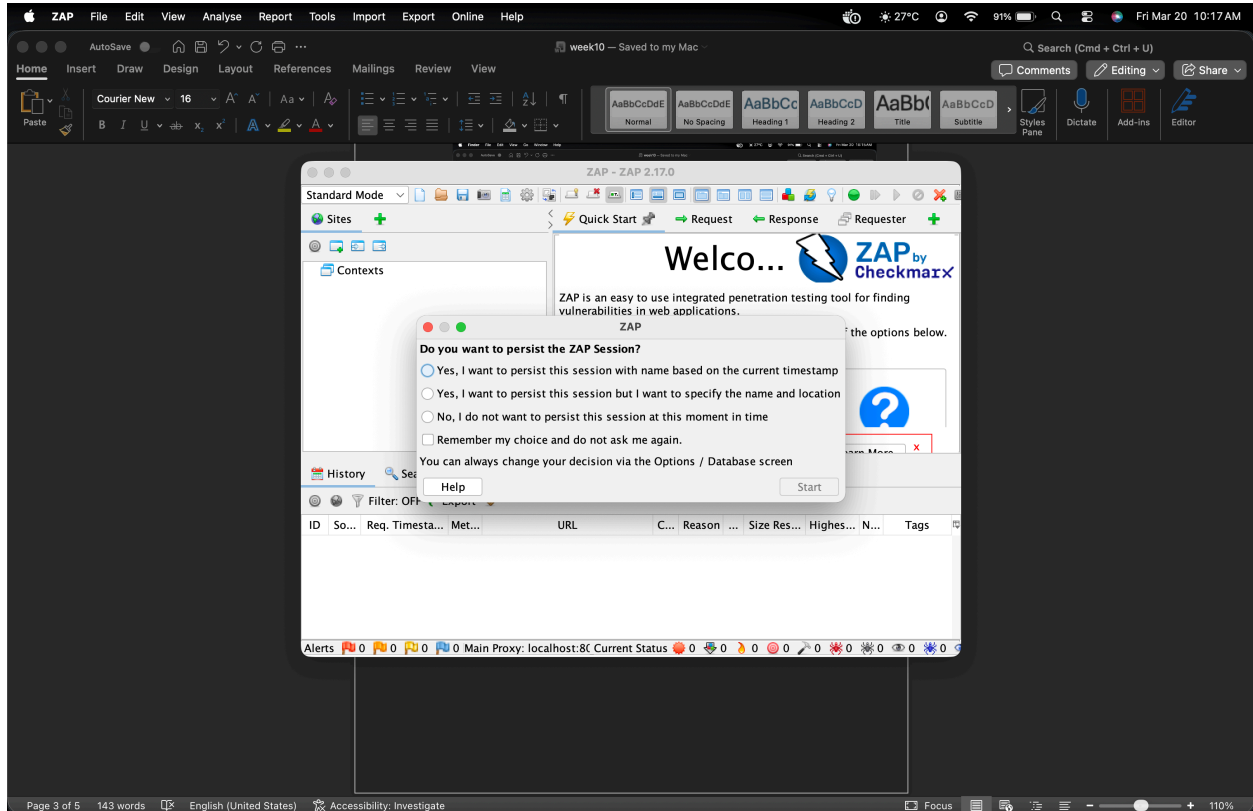
Step 1: Install OWASP ZAP on Windows

1. Go to the official site
<https://www.zaproxy.org/download/>
2. Download the **Windows Installer**
3. Run the installer and follow the setup.



After installation open:

OWASP ZAP



Step 2: Install Node.js

Juice Shop runs using **Node.js**.

1. Go to <https://nodejs.org>
2. Download **LTS version**
3. Install with default settings.

Step 3: Install OWASP Juice Shop

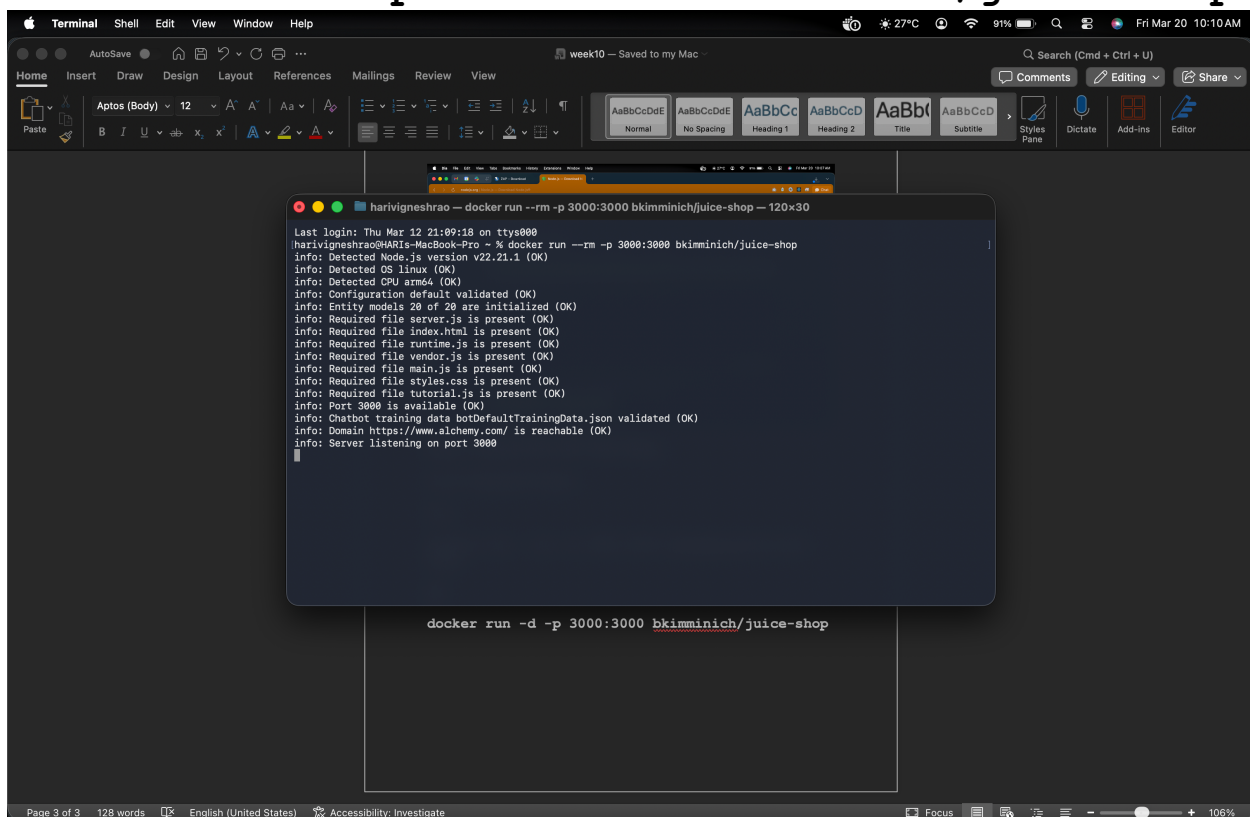
Open Command Prompt.

Run:

```
docker run --rm -p 3000:3000 bkimminich/juice-shop
```

OR

```
docker run -d -p 3000:3000 bkimminich/juice-shop
```



Go to the folder:

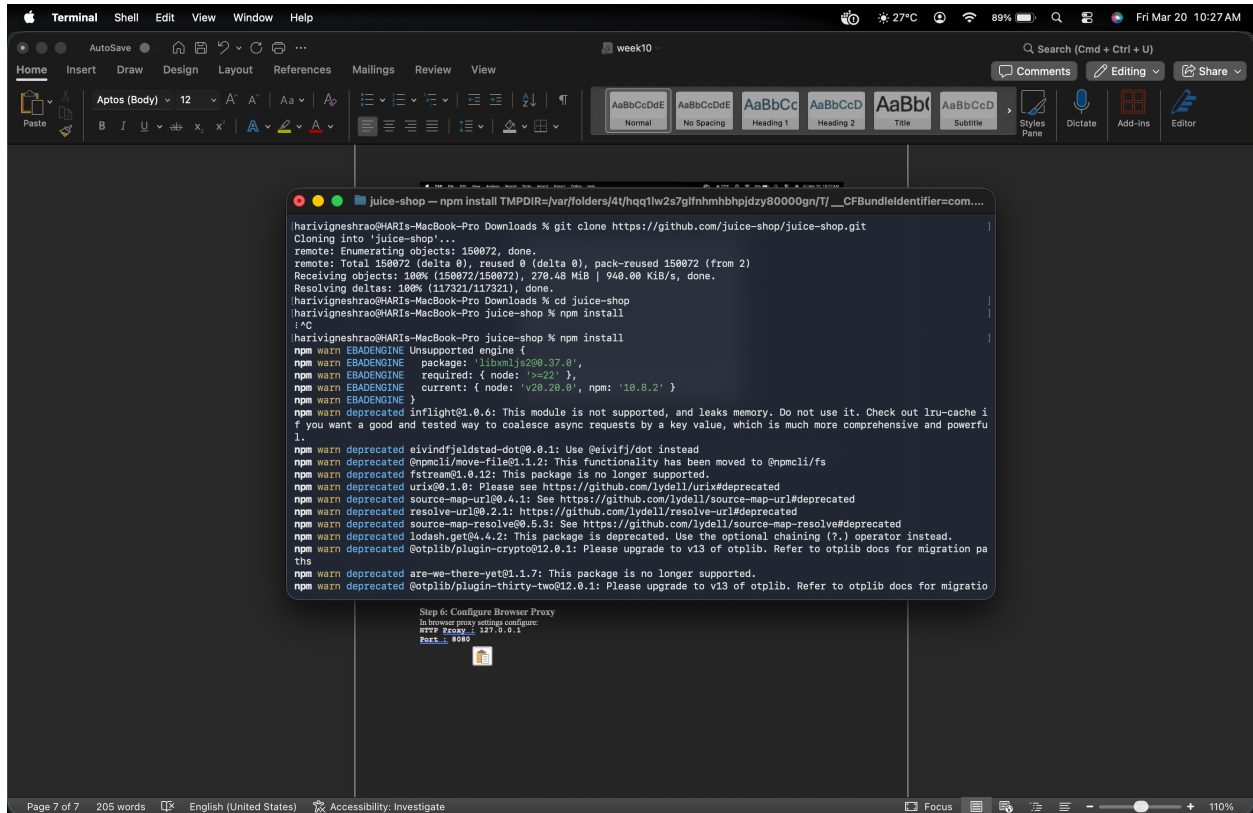
```
cd juice-shop
```

Install dependencies:

npm install

Start the application:

npm start

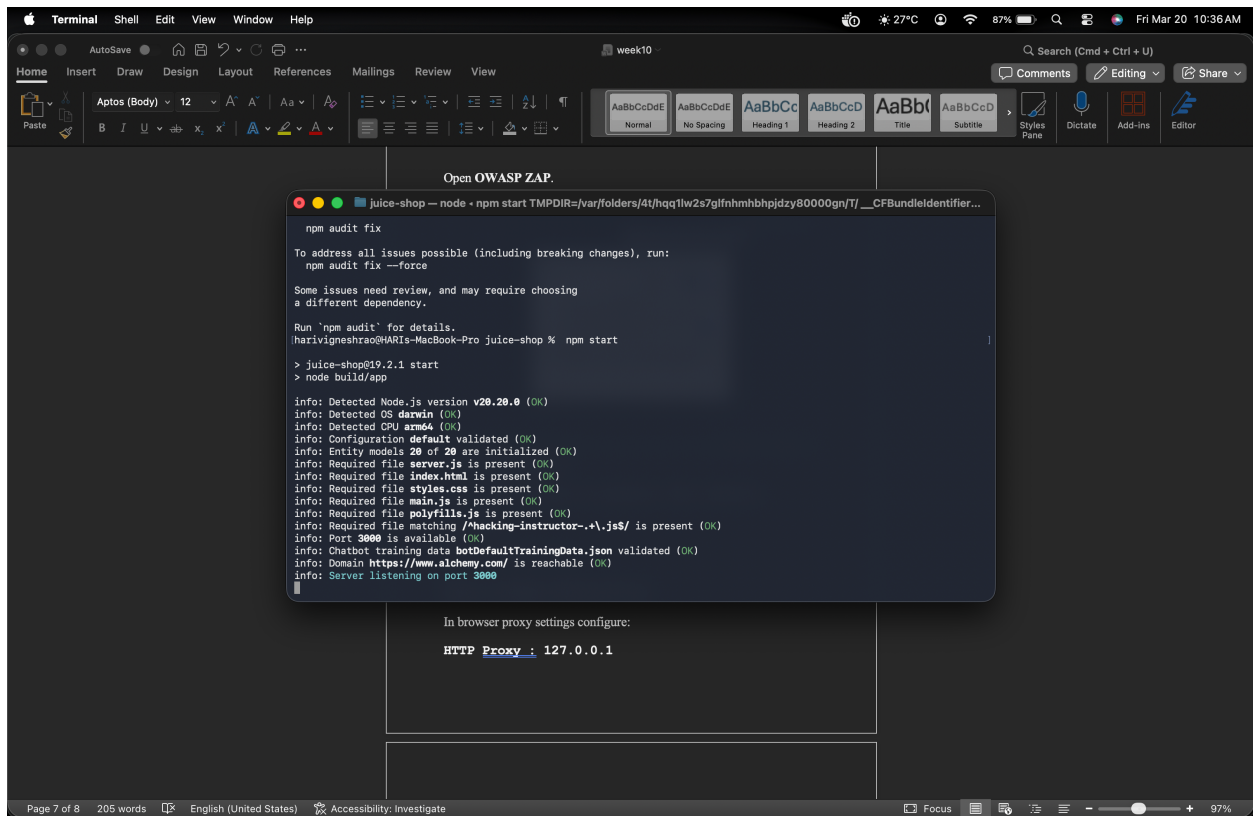


The screenshot shows a macOS Terminal window with the following content:

```
harivigneshrao@HARIS-MacBook-Pro Downloads % git clone https://github.com/juice-shop/juice-shop.git
Cloning into 'juice-shop'...
remote: Enumerating objects: 150072, done.
remote: Total 150072 (delta 0), reused 0 (delta 0), pack-reused 150072 (from 2)
Receiving objects: 100% (150072/150072), 270.48 MiB | 940.00 KiB/s, done.
Resolving deltas: 100% (117321/117321), done.
harivigneshrao@HARIS-MacBook-Pro Downloads % cd juice-shop
harivigneshrao@HARIS-MacBook-Pro juice-shop % npm install
npm warn EBADENGINE Unsupported engine {
  npm warn EBADENGINE   package: 'libxmljs2@0.37.0',
  npm warn EBADENGINE   required: { node: '>=22' },
  npm warn EBADENGINE   current: { node: 'v20.20.0', npm: '10.8.2' }
}
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if
  you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerfu
  l.
npm warn deprecated olivindfieldstd-dot@0.0.1: Use @olivifi/dot instead
npm warn deprecated @npmcli/move-file@1.1.2: This functionality has been moved to @npmcli/fs
npm warn deprecated fstream@1.0.12: This package is no longer supported.
npm warn deprecated urix@0.1.0: Please see https://github.com/lydell/urix#deprecated
npm warn deprecated source-map-url@0.4.1: See https://github.com/lydell/source-map-url#deprecated
npm warn deprecated resolve-url@0.2.1: https://github.com/lydell/resolve-url#deprecated
npm warn deprecated source-map-resolve@0.5.3: See https://github.com/lydell/source-map-resolve#deprecated
npm warn deprecated lodash.get@4.4.2: This package is deprecated. Use the optional chaining (?.) operator instead.
npm warn deprecated @otplib/plugin-crypto@12.0.1: Please upgrade to v13 of otplib. Refer to otplib docs for migration pa
  ttns
npm warn deprecated are-we-there-yet@1.1.7: This package is no longer supported.
npm warn deprecated @otplib/plugin-thirty-two@12.0.1: Please upgrade to v13 of otplib. Refer to otplib docs for migratio
  n

Step 6: Configure Browser Proxy
In browser proxy settings configure:
HTTP Proxy : 127.0.0.1
Port : 8080
```

The terminal output shows the successful cloning of the repository, installation of dependencies, and the start of the application. The application is running on port 8080.

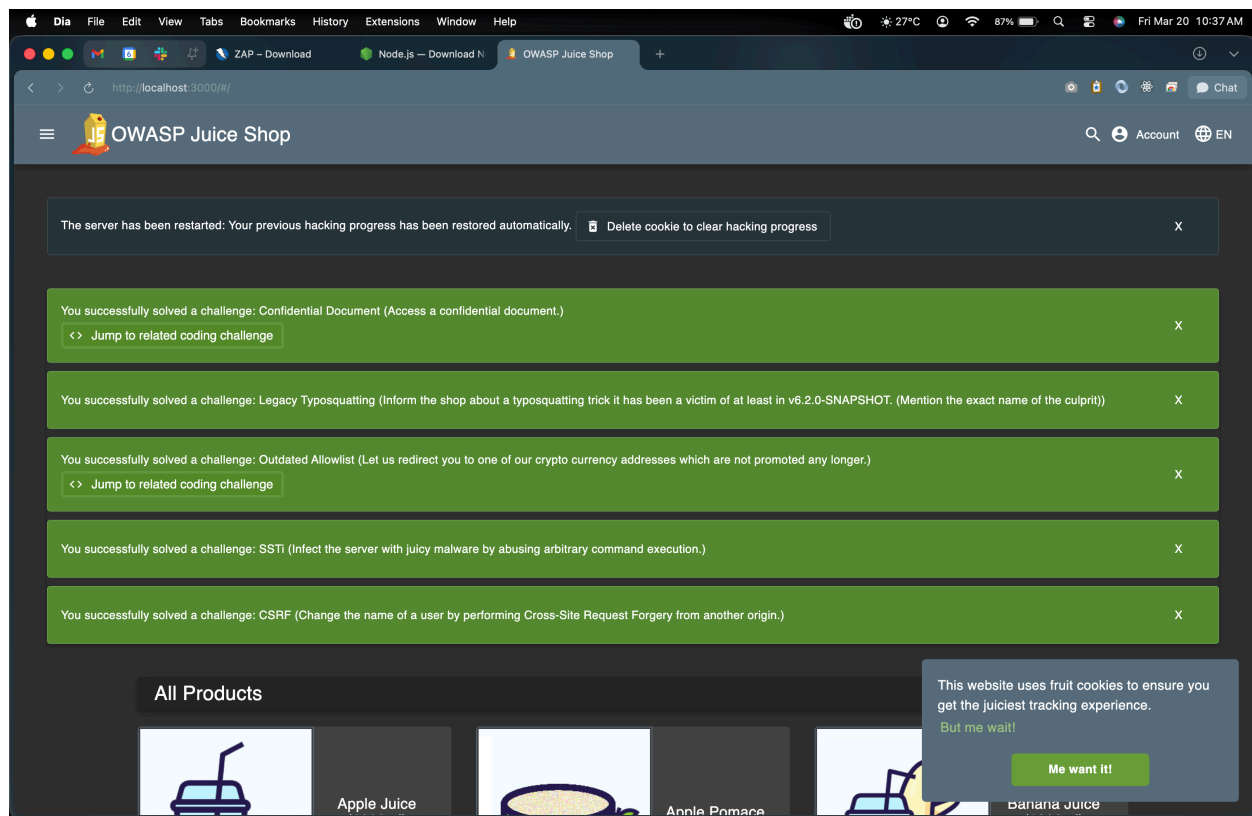


Step 4: Open Juice Shop

Open browser and go to:

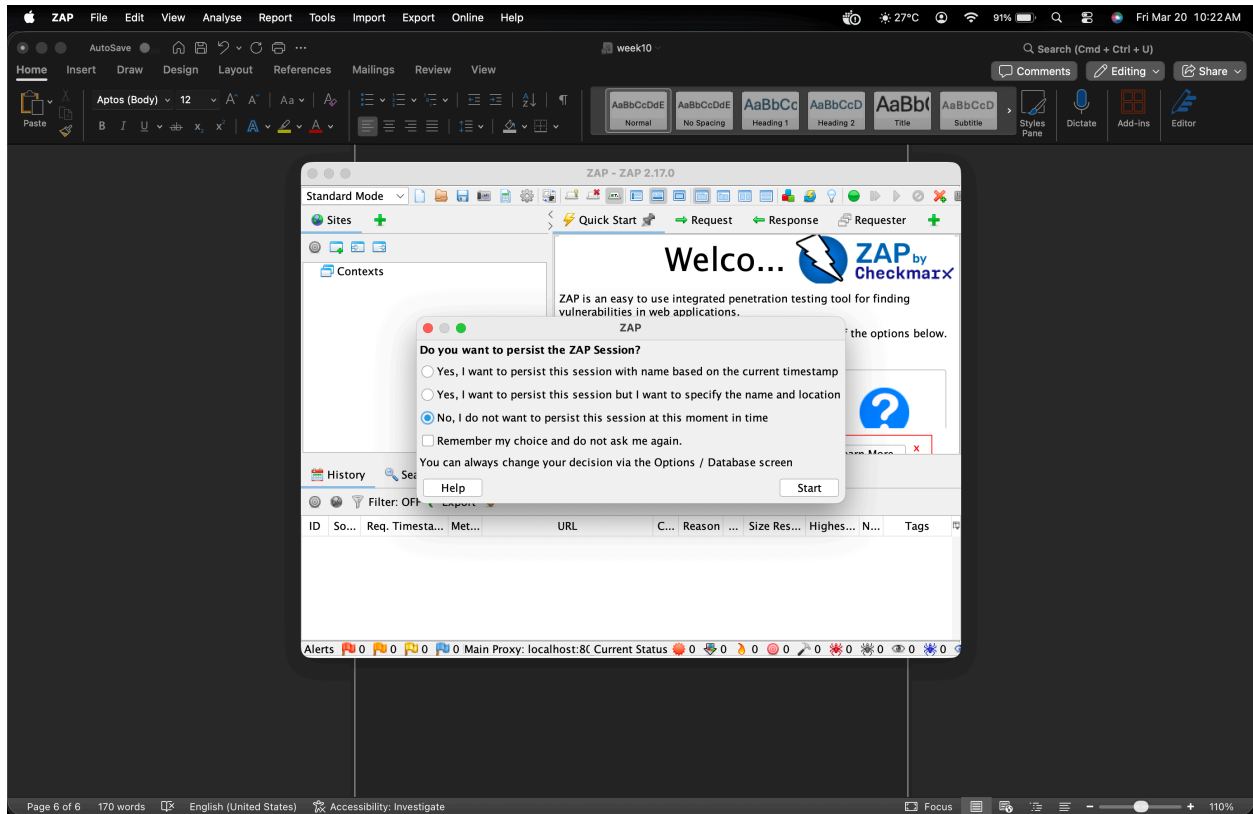
http://localhost:3000

You will see the **Juice Shop** web application.



Step 5: Start OWASP ZAP

Open **OWASP ZAP**.



When prompted choose:

No, I do not want to persist this session

Click **Start**.

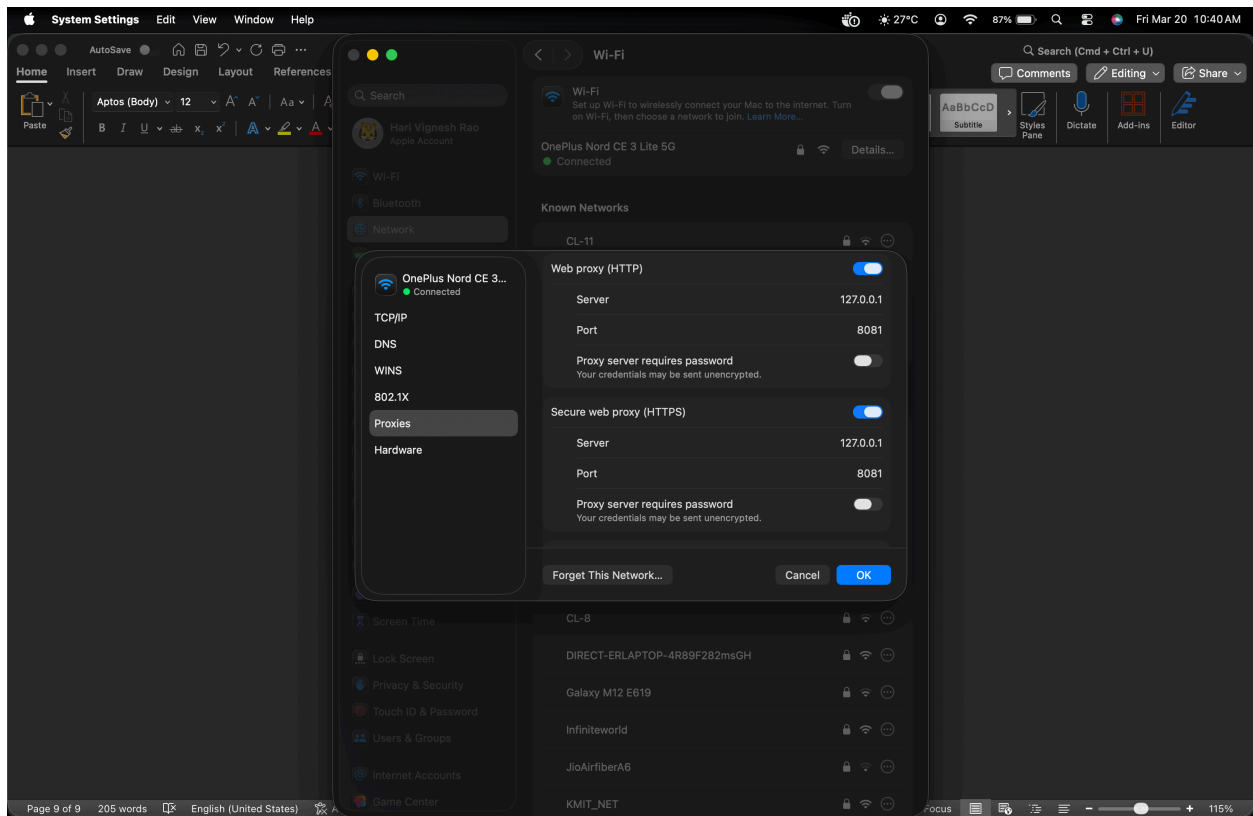
Click on automated scan

Step 6: Configure Browser Proxy

In browser proxy settings configure:

HTTP Proxy : 127.0.0.1

Port : 8080



Now browser traffic passes through **ZAP**.

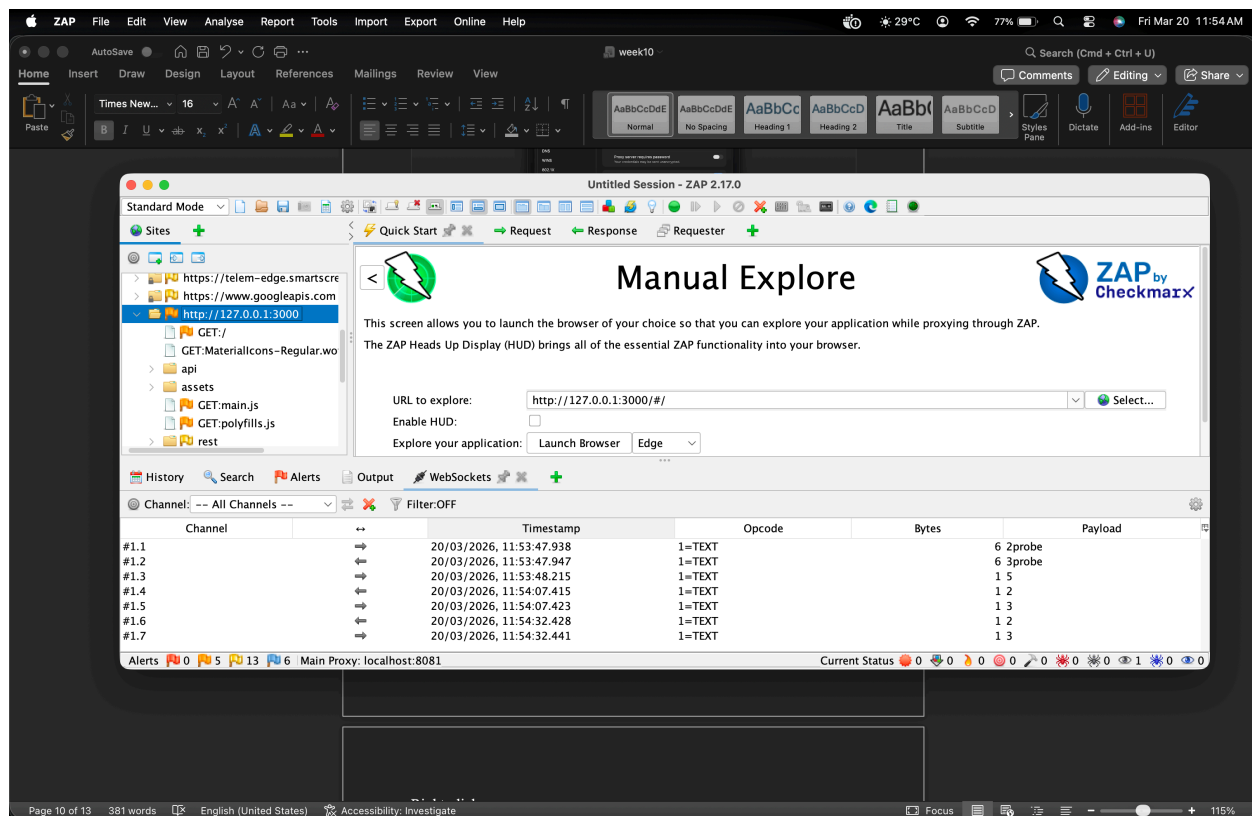
Step 7: Confirm Connection

Open:

`http://localhost:3000`

In ZAP **Sites** panel you will see:

`http://localhost:3000`



Step 8: Perform Spider Scan

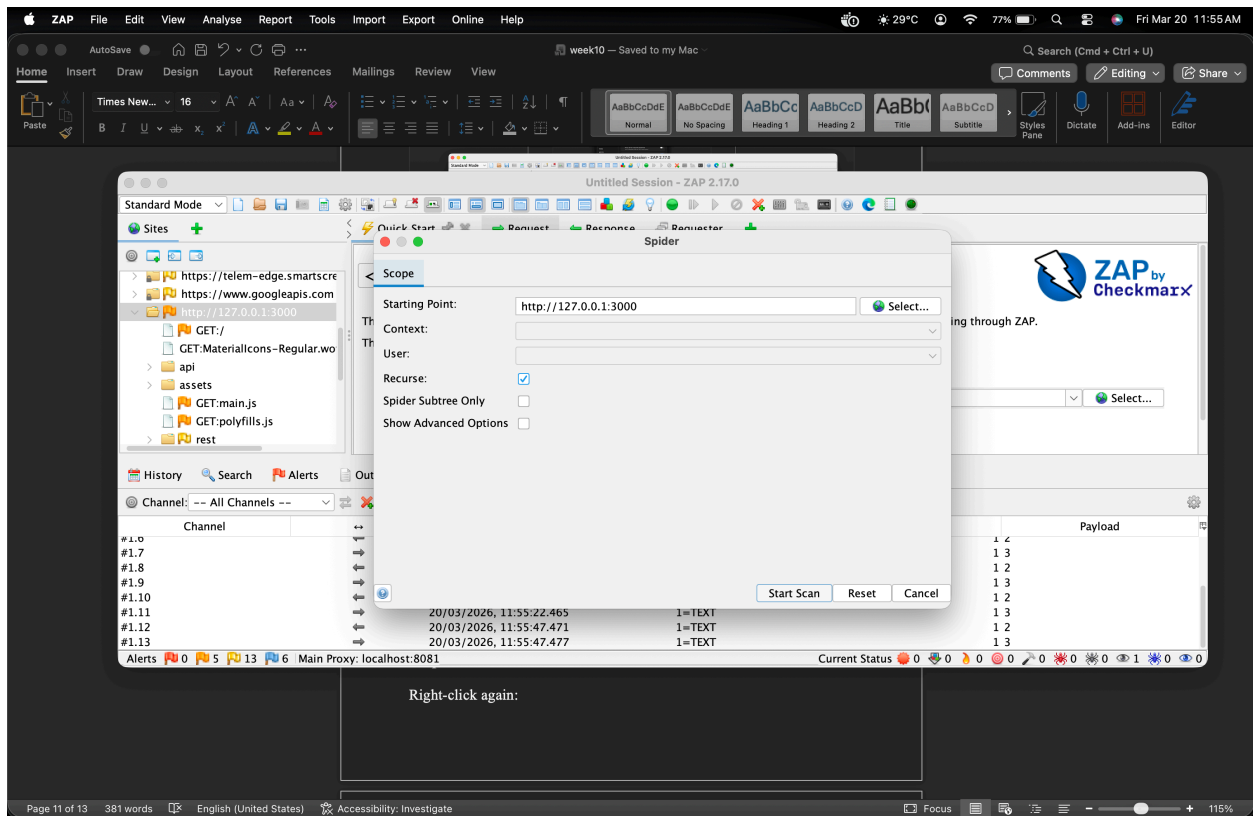
Right-click:

`http://localhost:3000`

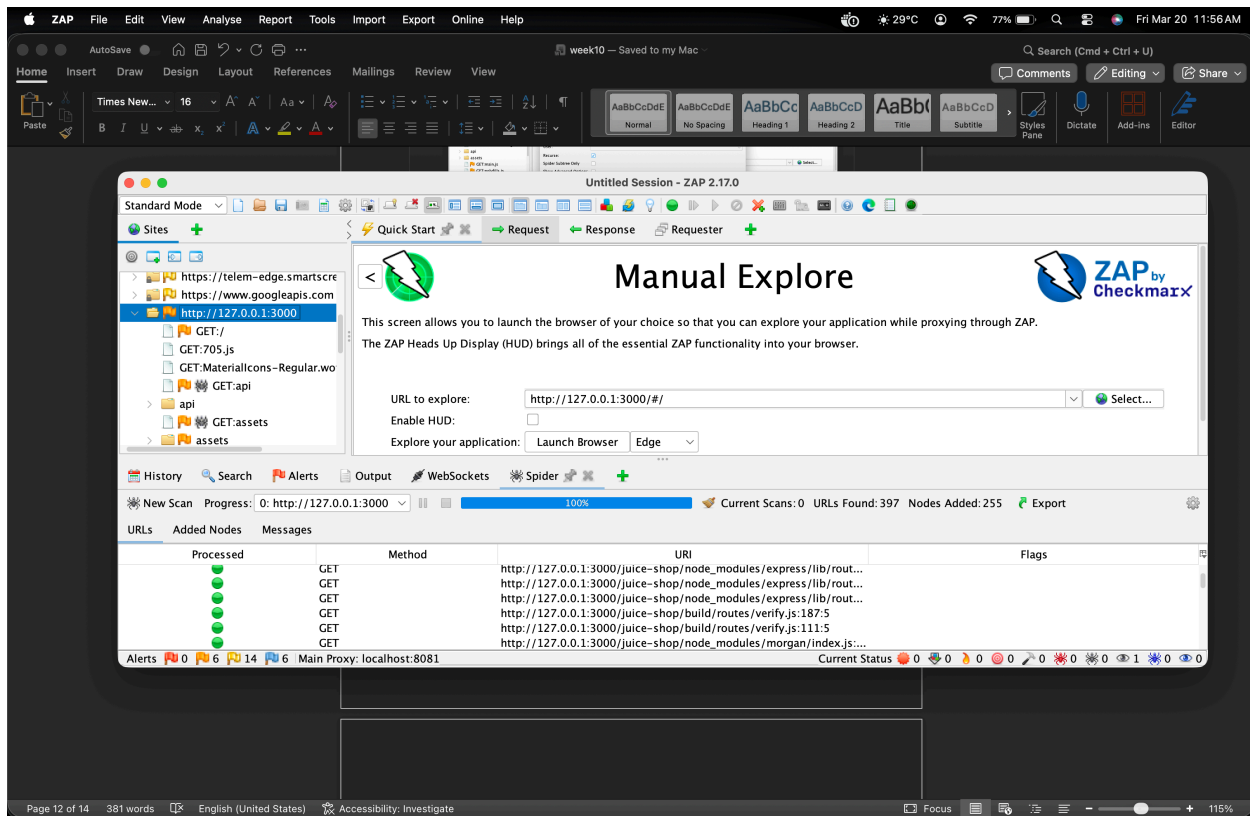
Select:

Attack → Spider

Click **Start Scan**.



ZAP will discover all pages.



Step 9: Perform Active Scan

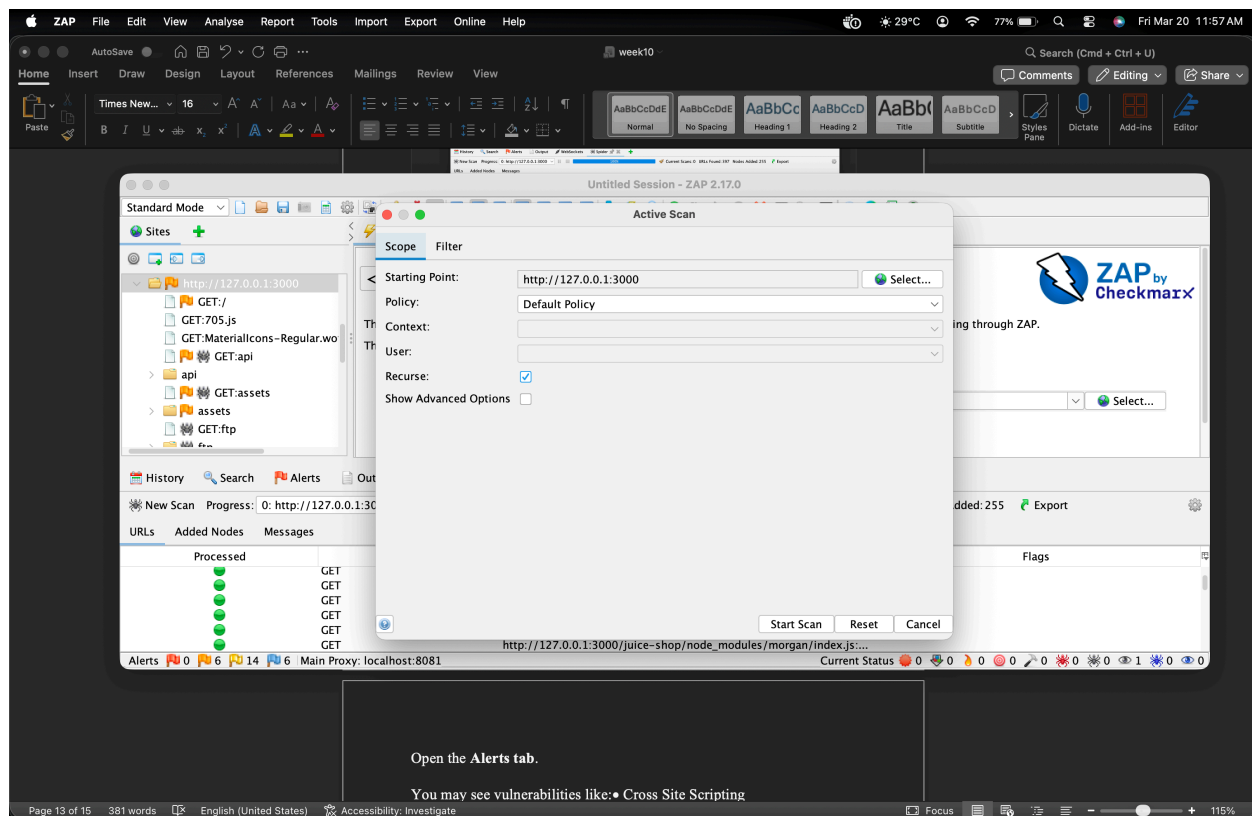
Right-click again:

http://localhost:3000

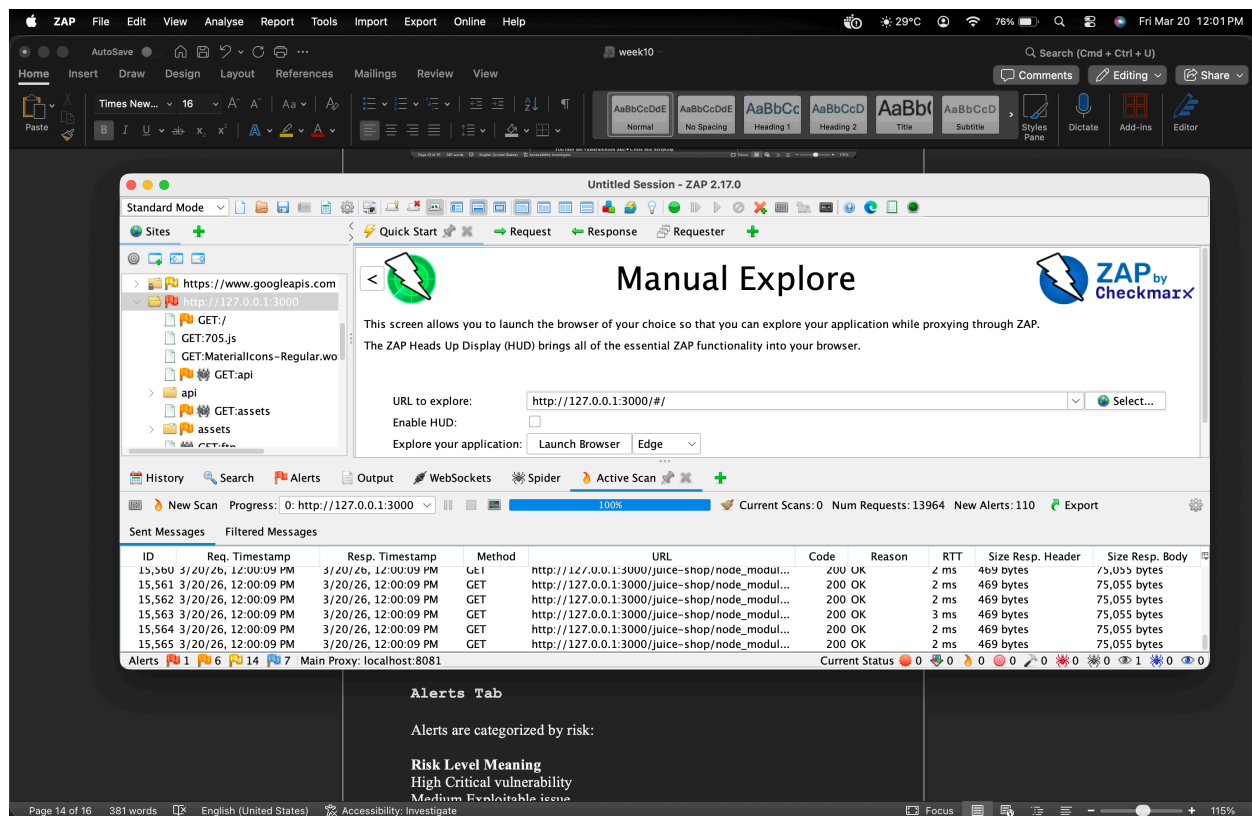
Select:

Attack → Active Scan

Click Start Scan.

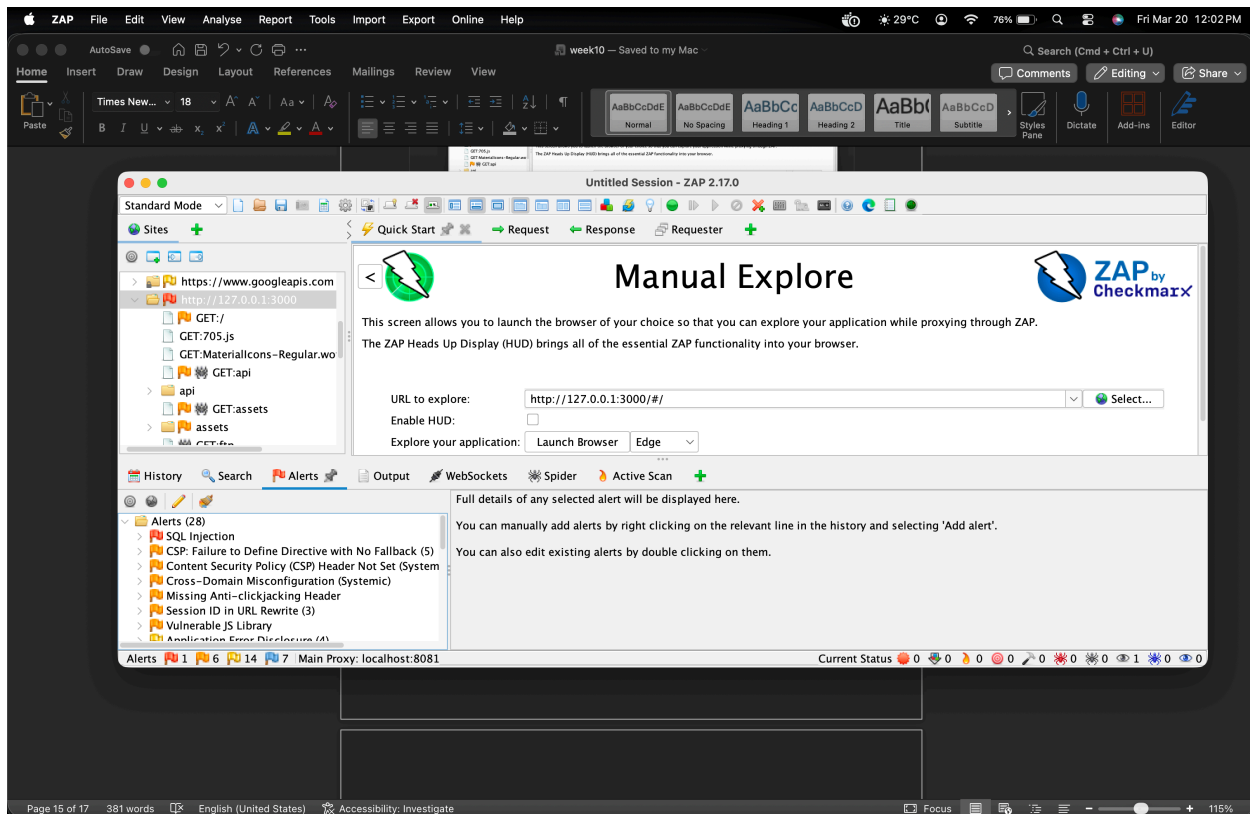


ZAP will test vulnerabilities.



Step 10: Analyze Vulnerabilities

Open the **Alerts** tab.



You may see vulnerabilities like: • Cross Site Scripting

- Missing security headers
- Cookie security issues
- Information disclosure

Analysing Results

Step 11: View Alerts

Go to:

Alerts Tab

Alerts are categorized by risk:

Risk Level	Meaning
------------	---------

High	Critical vulnerability
Medium	Exploitable issue
Low	Minor issue
Informational	No direct risk

Click any alert to view:

- Description
- Evidence
- Attack payload used
- Suggested remediation

Step 12: Export Scan Report

1. Go to **Report**
2. Choose format:
 - HTML
 - PDF
 - XML
3. Select:
 - Risk levels
 - Confidence level
4. Generate report
Save file.

Result

The vulnerable web application was scanned using **OWASP ZAP**, and multiple

vulnerabilities were identified in **OWASP Juice Shop**.